EXPERIMENT 4 – INFORMATION RETRIEVAL USING NLTK

AIM

To build a review retrieval system using text preprocessing, TF-IDF and cosine similarity.

1. Load Amazon Fine Food Reviews and retain the Text column.
2. Remove nulls and duplicate reviews; keep up to 10,000 rows.
3. Lowercase, remove punctuation/numbers, tokenize and remove stopwords.
4. Build TF-IDF vectors and display matrix shape/vocabulary size.
5. Retrieve the five reviews most similar to each query using cosine similarity.

In [ ]

Copy

```
import pandas as pd, re, nltk
from nltk.corpus import stopwords
from nltk.tokenize import wordpunct_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
nltk.download(class="s">"stopwords",quiet=True)
df=pd.read_csv(class="s">"Reviews.csv")[[class="s">"Text"]].dropna().drop_dupli
stop=set(stopwords.words(class="s">"english"))
def clean(t):
 t=re.sub(rclass="s">"[^a-zA-Z\\s]",class="s">" ",str(t).lower())
 return class="s">" ".join(w for w in wordpunct_tokenize(t) if w not in stop an
df[class="s">"Cleaned_Text"]=df.Text.apply(clean); print(df.head())
vec=TfidfVectorizer(); M=vec.fit_transform(df.Cleaned_Text)
print(class="s">"TF-IDF shape:",M.shape,class="s">"Vocabulary:",len(vec.vocabul
def retrieve(q,k=5):
 sim=cosine_similarity(vec.transform([clean(q)]),M).ravel(); ix=sim.argsort()[-
 return df.iloc[ix][[class="s">"Text",class="s">"Cleaned_Text"]].assign(Similar
for q in [class="s">"great product with fast shipping",class="s">"disappointed"
 print(class="s">"QUERY:",q); print(retrieve(q).to_string(index=False))
```

RESULT

Thus, the system retrieves relevant customer reviews for natural-language queries using TF-IDF and cosine similarity.